Claude Tool Use

Function Calling, Computer Use & Agentic Patterns

Anthropic Official Documentation | docs.anthropic.com/en/docs/build-with-claude/tool-use

---

Overview

Claude supports a rich tool-use (function calling) system that allows it to call external functions, APIs, and services. This document covers the official tool-use API from Anthropic's documentation, including best practices, patterns, and examples.

Source: docs.anthropic.com/en/docs/build-with-claude/tool-use

1. How Tool Use Works

Claude can be given a list of tool definitions at the start of a conversation. When Claude determines a tool call would help respond to the user, it outputs a tool_use block. The client executes the function and returns the result as a tool_result block. Claude then incorporates the result into its response.

```
The tool-use loop: User → Claude (decides to use tool) → tool_use block → Client executes → tool_result → Claude (final answer)
```

2. Tool Definition Format

```
{
  "name": "get_weather",
  "description": "Get current weather for a location. Call this when the user asks a bout weather.",
  "input_schema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City and country, e.g. 'San Francisco, US'"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "Temperature unit"
      }
    },
    "required": ["location"]
  }
```

```
}
```

3. Tool Choice Options

tool_choice	Behavior
`{"type": "auto"}`	Claude decides whether to use tools (default)
`{"type": "any"}`	Claude must use at least one tool
`{"type": "tool", "name": "X"}`	Claude must use the specific tool X
`{"type": "none"}`	Claude cannot use any tools

4. Complete Python Example

```
import anthropic

client = anthropic.Anthropic()

tools = [{
    "name": "search_documents",
    "description": "Search a knowledge base for relevant documents",
    "input_schema": {
        "type": "object",
        "properties": {
            "query": {"type": "string", "description": "Search query"},
            "max_results": {"type": "integer", "description": "Max results (1-10)"}
        },
        "required": ["query"]
    }
}]

def search_documents(query, max_results=5):
    # Your search implementation
    return {"results": [{"text": f"Result for {query}", "score": 0.95}]}

messages = [{"role": "user", "content": "Find docs about vector databases"}]

# Agentic loop
while True:
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        tools=tools,
        messages=messages
    )
```

```
if response.stop_reason == "end_turn":
    print(response.content[0].text)
    break

# Process tool calls
messages.append({"role": "assistant", "content": response.content})
tool_results = []

for block in response.content:
    if block.type == "tool_use":
        result = search_documents(**block.input)
        tool_results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": str(result)
        })

messages.append({"role": "user", "content": tool_results})
```

5. Computer Use (Beta)

Claude 3.5 Sonnet introduced computer use — the ability to control a computer through screenshot + action tools: computer (screenshot, click, type), bash (run commands), text_editor (view/edit files). This enables fully autonomous coding and browser agents.

6. Best Practices

- Write clear tool descriptions — Claude relies entirely on the description to decide when to call
- Use required fields minimally; give Claude flexibility to omit optional params
- For multi-step workflows, prefer a single orchestrator Claude + specialized subagents
- Implement tool timeouts and error handling — always return tool_result even on failure
- Use parallel tool use (betas=["tools-2024-04-04"]) for independent concurrent calls
- Log all tool calls and results for debugging and cost tracking